Pontificia Universidad Católica de Chile
Escuela de Ingeniería
IIC-3694 Inteligencia Artificial Generativa

Efecto del SFT previo en RL para razonamiento con unidades

Proyecto de investigación · Entrega 1: propuesta

Fernando Fajardo, Sebastian Palma, Benjamin Prieto y Rodrigo Ugarte · 11 de noviembre de 2026

1. Problema y pregunta de investigación

El proyecto estudiará cómo mejorar el modelo de lenguaje Qwen3-1.7B para resolver problemas cuantitativos con unidades mediante entrenamiento adicional. Para ello, usaremos el entorno construido en el desafío del curso, que genera tareas en español y verifica sus resultados automáticamente. Sobre esta base compararemos ajuste supervisado (SFT), aprendizaje por refuerzo (RL) con recompensas verificables y optimización directa de preferencias (DPO).

La pregunta principal es: *¿cómo influye el SFT previo en la eficiencia del aprendizaje con RL bajo un presupuesto de cómputo comparable?* Para responderla, contrastaremos SFT seguido de RL con solo RL, e incluiremos solo SFT, DPO y el modelo inicial como referencias. El SFT podría facilitar que el modelo produzca soluciones correctas y reciba recompensa durante RL, aunque también podría consumir recursos sin mejorar su desempeño en problemas nuevos.

Nuestra hipótesis es que el SFT previo mejora el rendimiento en validación por unidad de cómputo frente a iniciar directamente con RL. Evaluaremos si esa ventaja persiste al incluir el costo del SFT y si refleja mayor exactitud numérica o solo un mejor cumplimiento del formato. Una auditoría manual complementará estas medidas para examinar la calidad de las soluciones.

2. Antecedentes y relación con la propuesta

La generación automática de tareas ofrece un punto de partida para esta comparación. Stojanovski et al. (2025) presentan Reasoning Gym [1], una biblioteca de tareas generadas por código con dificultad controlable y verificación automática. Este diseño motiva el uso de nuestro entorno para producir recompensas y estudiar el aprendizaje por dificultad. Su aplicación requerirá comprobar que nuestros datos tengan suficiente volumen y una mezcla de aciertos y errores que permita aprender.

La evaluación debe considerar, además, la sensibilidad a las variantes de una tarea. GSM-Symbolic, de Mirzadeh et al. (2024) [2], muestra variación del desempeño al cambiar valores y condiciones de problemas matemáticos. Esto motiva conservar variantes relacionadas en la misma partición y evaluar por familia y nivel, evitando atribuir generalización a un promedio global. En nuestro diseño inicial se compartirán fórmulas entre particiones, por lo que la generalización se evaluará sobre nuevos valores y representaciones dentro de las familias elegidas.

En cuanto al entrenamiento, Rafailov et al. (2023) [3] proponen DPO, que aprende de pares de respuestas preferidas y rechazadas sin ajustar un modelo explícito de recompensa ni generar nuevas respuestas durante la actualización. Esto ofrece una alternativa a RL cuya eficacia evaluaremos en tareas con verificación numérica. Los pares se construirán a partir de respuestas verificadas y revisadas, aunque su cantidad inicial es reducida.

Para las ramas de RL, DeepSeekMath, de Shao et al. (2024) [4], introduce GRPO, un algoritmo que compara las recompensas de varias respuestas al mismo problema y prescinde del modelo de valor adicional de PPO. Sus resultados de RL parten de un modelo previamente ajustado con SFT, de modo que no resuelven directamente nuestra comparación con RL desde el modelo inicial. Por ello proponemos GRPO para ambas ramas, sujeto a comprobar su viabilidad en el hardware disponible.

Finalmente, Aleithan et al. (2024) [5] muestran mediante revisión manual que superar pruebas automáticas de programación puede ocultar soluciones problemáticas. Esta observación motiva acompañar la evaluación

automática con una auditoría humana de respuestas exitosas. Así podremos examinar si los aciertos registrados corresponden a soluciones válidas en nuestro entorno.

3. Entorno y datos disponibles

El entorno del desafío contiene tres familias, entendidas como tipos de problema definidos por una fórmula: velocidad $v = d/t$, energía cinética $E = mv^2/2$ y luminosidad $L = 4\pi d^2 F$. Se proporcionan fórmula y equivalencias, por lo que el modelo debe componer operaciones y conversiones sin recordar constantes ni utilizar herramientas. Los datos pueden darse directamente o como sumas de distancias, masas o flujos. Cada problema tiene tres niveles de dificultad: el fácil no exige conversiones, el medio requiere convertir una entrada y el difícil requiere convertir dos. En el nivel difícil de velocidad y energía también se pide convertir la salida.

Los 60 problemas físicos, con sus tres variantes, generan $60 \times 3 = 180$ tareas. Por cada familia se asignan 10 problemas a entrenamiento, cuatro a validación y seis a prueba, manteniendo juntas sus variantes. Esto produce 90, 36 y 54 tareas respectivamente. Por ejemplo, una tarea proporciona distancias de 4,08 y 1,02 km y un tiempo de 88 s, y pide $(d_1 + d_2)/t$ en m/s. La salida `57.95 m/s` es aceptable frente al resultado esperado 5100/88 m/s.

El verificador recibe el enunciado y la respuesta mediante la función `reward(prompt, completion)`. Devuelve 1 si extrae un único resultado finito, con unidades compatibles y un error relativo de hasta el 1 % tras convertirlo al Sistema Internacional. En caso contrario devuelve 0. Las referencias se calculan por código y no se envían al modelo. El extractor puede rechazar formatos correctos y actualmente admite unidades equivalentes distintas de la solicitada, por lo que su recompensa no certifica todos los pasos ni obliga siempre a realizar la conversión de salida.

Disponemos de 121 soluciones SFT generadas por Qwen3-8B y 35 pares DPO, todos de entrenamiento y con procedencia registrada. Agentes de IA revisaron estos datos, que aún requieren revisión humana. Además, los niveles difíciles de energía y luminosidad cuentan con solo una solución SFT cada uno. Por ello, revalidaremos y, si es necesario, ampliaremos los datos antes de fijar los experimentos.

Como antecedente, Qwen3-1.7B alcanzó 51,85 % en pass@1 y 68,52 % en pass@3 en el desafío. El extractor pudo interpretar el 95,68 % de las respuestas. Esta línea base se obtuvo con Ollama y pesos cuantizados en formato GGUF, y se repetirá con la configuración que se utilice para entrenar.

4. Comparación experimental propuesta

Con este punto de partida, proponemos un estudio de métodos con implementaciones públicas, correspondiente al nivel de complejidad 1 de la pauta. Las cinco rutas partirán de los mismos pesos iniciales de Qwen3-1.7B y compartirán enunciados, particiones y protocolo de evaluación. La tabla resume el entrenamiento de cada ruta.

Ruta	Procedimiento
Control	Evaluar el modelo inicial sin ajuste adicional.
Solo SFT	Ajustar con las soluciones correctas de entrenamiento.
SFT $\rightarrow$ RL	Aplicar SFT y continuar con GRPO y recompensa del verificador.
Solo RL	Aplicar GRPO desde el modelo inicial, sin SFT propio previo.
DPO	Ajustar con pares de respuestas preferidas y rechazadas, manteniendo el modelo inicial como referencia fija.

Primero realizaremos un piloto para medir memoria y tiempo. Con sus resultados fijaremos un presupuesto B en horas de GPU por ruta, usando el mismo hardware. SFT seguido de RL contabilizará ambas fases dentro de B, mientras que el control solo requerirá evaluación. Registraremos también los tokens procesados, el costo de las evaluaciones y el costo compartido de generar datos. Esto permitirá comparar el beneficio de cada estrategia a igual presupuesto de entrenamiento.

El análisis de sensibilidad variará la fracción de B dedicada al SFT previo: compararemos 25 % y 50 %, dejando el resto para RL, si el piloto permite ejecutar ambas condiciones. Mantendremos comunes los demás hiperparámetros de RL para aislar el efecto del SFT previo. Las versiones del modelo obtenidas durante el entrenamiento se seleccionarán con validación, sin usar el conjunto de prueba.

El diseño es tentativo y podrá ajustarse según el piloto y el análisis de errores. Entre las iteraciones previstas están ampliar las familias de problemas, reequilibrar los niveles de dificultad o explorar variantes de la recompensa. Estas extensiones se priorizarán según los recursos disponibles, manteniendo la pregunta central y la comparación de las cinco rutas. Todo cambio se documentará y dará lugar a una evaluación común de las rutas afectadas, conservando la separación de datos por problema físico.

5. Evaluación, recursos y próximos hitos

Para evaluar las cinco rutas, mediremos pass@1 y pass@3 por familia y nivel, generando tres respuestas por tarea. Pass@1 estima la proporción de respuestas correctas y pass@3 la proporción de tareas resueltas al menos una vez entre tres intentos. También registraremos la proporción de respuestas interpretables y los errores numéricos para distinguir capacidad de cálculo de cumplimiento del formato. Usaremos la misma configuración de muestreo, con semillas de generación 42–44 y límites fijados con validación.

La hipótesis se contrastará con curvas de pass@1 en validación frente al cómputo acumulado, incluyendo el SFT previo, y con el rendimiento final a igual B. Las curvas de recompensa durante RL permitirán detectar falta de señal de aprendizaje. Como complemento, se auditará manualmente una muestra de hasta tres éxitos por familia, nivel y ruta, elegidos aleatoriamente cuando existan. Se revisarán datos, conversiones, operaciones y conclusión. Para cada grupo se informarán el número de respuestas revisadas y la proporción de aciertos genuinos, junto con la recompensa automática original.

Antes de entrenar se precisarán y probarán las políticas de formatos, unidades y respuestas contradictorias. Si cambia la recompensa, se revalidarán los datos de SFT y DPO. Además, reservaremos nuevos problemas para la prueba final, ya que los del desafío se utilizaron para diagnosticar limitaciones. Todos los resultados finales se evaluarán con la misma versión final del verificador y se documentarán los problemas detectados, las correcciones y su efecto al reentrenar. La dependencia entre variantes y los pocos ejemplos en algunas combinaciones de familia y dificultad limitarán las conclusiones estadísticas.

El recurso inicial confirmado es una RTX 3050 Laptop con 6 GiB de memoria de GPU. El piloto probará adaptadores LoRA, que permiten entrenar un conjunto reducido de parámetros, y limitará la longitud de las respuestas. La viabilidad de RL con esta memoria sigue pendiente de comprobación. Otro colaborador podría aportar cómputo, aunque su disponibilidad aún no está confirmada. Si se cambia de GPU, repetiremos el piloto y fijaremos condiciones equivalentes para las cinco rutas. Las repeticiones con distintas semillas de entrenamiento dependerán del cómputo disponible.

Para el avance del 25 de noviembre se prevén el piloto, la adaptación del entorno, la nueva línea base y las primeras curvas de recompensa. Para el cierre del 9 de diciembre se prevé completar las cinco rutas, el análisis de sensibilidad y la auditoría, junto con el artículo, el póster y su presentación en video. Estos resultados permitirán evaluar cuándo conviene dedicar parte del presupuesto al SFT antes de RL y qué limitaciones condicionan esa decisión.

Referencias

[1] Z. Stojanovski, O. Stanley, J. Sharratt, R. Jones, A. Adefioye, J. Kaddour y A. Köpf. Reasoning Gym: Reasoning Environments for Reinforcement Learning with Verifiable Rewards. 2025. https://arxiv.org/abs/2505.24760.

[2] I. Mirzadeh, K. Alizadeh, H. Shahrokhi, O. Tuzel, S. Bengio y M. Farajtabar. GSM-Symbolic: Understanding the Limitations of Mathematical Reasoning in Large Language Models. 2024. https://arxiv.org/abs/2410.05229.

[3] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning y C. Finn. Direct Preference Optimization: Your Language Model Is Secretly a Reward Model. 2023. https://arxiv.org/abs/2305.18290.

[4] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu y D. Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. 2024. https://arxiv.org/abs/2402.03300.

[5] R. Aleithan, H. Xue, M. M. Mohajer, E. Nnorom, G. Uddin y S. Wang. SWE-Bench+: Enhanced Coding Benchmark for LLMs. 2024. https://arxiv.org/abs/2410.06992.